

Table des matières

Introduction

Système d'exploitation

Ligne de commande

Processus et fils d'exécution

Utilisateurs et groupes

Fichiers

Gestion de la mémoire

Concurrence et parallélisme

Compilation

Environnement logiciel et virtualisation

Rappels

Objectifs fondamentaux du noyau

- 1 répartir équitablement les ressources
- 2 garantir la sécurité des données

Processus

- instance d'un programme à l'instant t
- dispose d'un espace mémoire et des ressources système qui lui sont propres

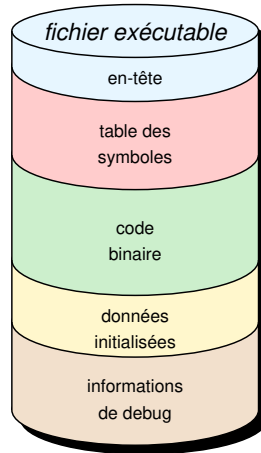
Questions

- Comment faire cohabiter plusieurs processus en mémoire tout en garantissant la sécurité des données ?
- Comment attribuer de la mémoire à tous les processus ?
- Comment exécuter des processus plus grands que la mémoire ?

Fichier exécutable

Aperçu de la structure

- format spécifique lisible par le lanceur de processus
 - *Executable and Linkable Format* (ELF)
 - découpage en sections
 - en-tête avec toutes les informations nécessaires à l'architecture
 - description dans le manuel avec `man elf`



Fichier exécutable

Exemple

/bin/ls : en-tête de l'exécutable (tronquée)

```
readelf -h /bin/ls | head -n 10
```

ELF Hlavička:

```

Kúzlo:  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
Trieda:                                ELF64
Dáta:                                     komplement 2, malý endián
Version:                               1 (current)
OS / ABI:                              UNIX - System V
ABI verzia:                             0
Typ:                                    DYN (Position-Independent Executable file)
Počítač:                               Advanced Micro Devices X86-64
Verzia:                                 0x1

```

/bin/ls : contenu de l'exécutable (32 premiers octets)

```
hexyl --border ascii --color never --length 32 /bin/ls
```

```

+-----+-----+-----+-----+-----+
|00000000| 7f 45 4c 46 02 01 01 00 | 00 00 00 00 00 00 00 00 | ●ELF●●●0|00000000|
|00000010| 03 00 3e 00 01 00 00 00 | 30 6d 00 00 00 00 00 00 | ●0>0●000|0m000000|
+-----+-----+-----+-----+-----+

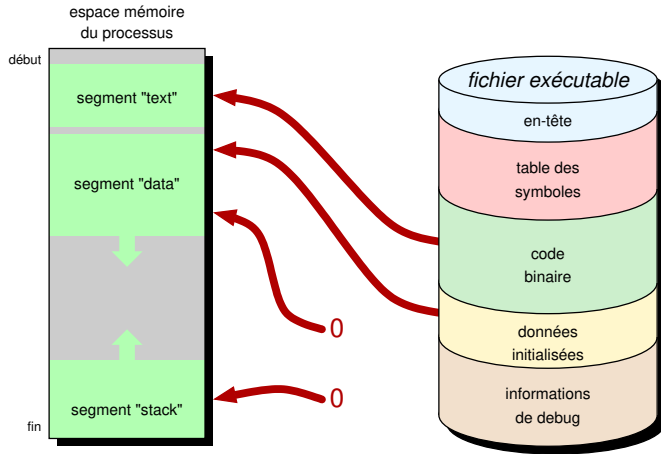
```

Lanceur de processus

- 1 invocation de `execv*()`
- 2 détection du type de l'exécutable
- 3 **projection de l'exécutable en mémoire**
- 4 préparation de l'interpréteur (si nécessaire)
- 5 passage du contrôle au programme
- 6 chargement des bibliothèques en mémoire
- 7 invocation de `start()`
- 8 invocation de `__libc_start_main()(_init & _fini)`
- 9 invocation de `main()`

Espace mémoire d'un processus

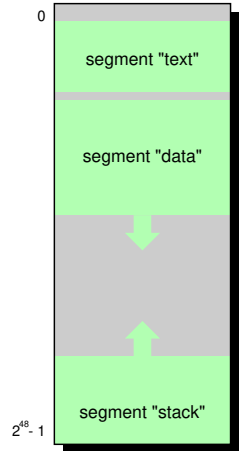
Projection en mémoire



Espace mémoire d'un processus

Sur une architecture à 64 bits

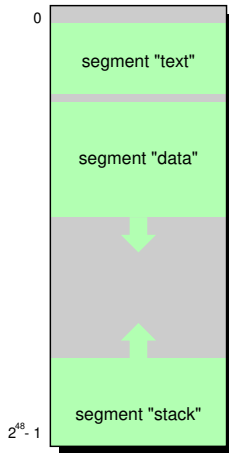
- vision unique de la mémoire par chaque processus
- 64 bits théoriques, seulement 48 bits sont câblés
- 2^{48} , c'est-à-dire 256 Tio adressables



Espace mémoire d'un processus

Pile

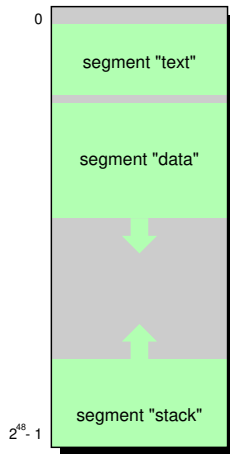
- partie de la mémoire du processus dans le segment *stack*
- taille fixée au démarrage du programme par le système d'exploitation
 - par défaut, 8 Mio sur Linux (voir la commande `ulimit`)
- superposition de couches
 - créées à chaque appel à fonction
 - détruites à chaque retour de fonction
- composition d'une couche
 - arguments de la fonction appelée
 - variables locales
 - adresse de retour dans la fonction appelante
 - pointeur de pile de la couche précédente



Espace mémoire d'un processus

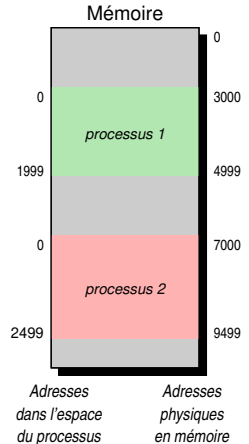
Tas

- partie de la mémoire du processus dans le segment *data*
- extensible en cours d'exécution du programme
- mémoire dynamique et persistante
- gérée par le programme lui-même
 - `malloc`, `calloc`, `realloc`
 - `free`



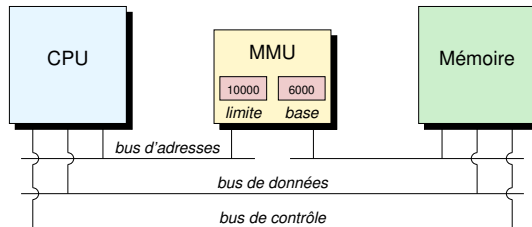
Espace d'adressage d'un processus

- chaque processus dispose de son propre espace d'adressage
- les programmes utilisent des adresses logiques relatives à l'espace d'adressage du processus



Traduction d'adresses

Unité de gestion de la mémoire :



Memory Management Unit (MMU)

- composant matériel
- traduction d'adresses logiques en adresses physiques
- vue linéaire et continue de la mémoire pour tous les programmes
- espaces d'adressage individuels et isolés pour chaque processus

Évincement de la mémoire

Que se passe-t-il lorsque la capacité de la mémoire physique ne suffit plus ?

Évincement de la mémoire

Que se passe-t-il lorsque la capacité de la mémoire physique ne suffit plus ?

- évicement des données d'un ou plusieurs processus de la mémoire vive vers le disque
- choix basé sur le temps de résidence en mémoire vive et l'état du processus

Espace d'échange

- espace ou fichier réservé sur le disque, également appelé *swap*

```
$ cat /etc/fstab
# <file system>          <mount point>  <type>      <options>      <dump>  <pass>
/dev/disk/by-id/dm...    /              ext4        defaults        0        1
...
/swap.img                none           swap        sw              0        0
...
```

Mémoire virtuelle

Motivation

Que se passe-t-il si l'ensemble de l'espace d'adressage d'un processus ne rentre pas dans la mémoire physique ?

Mémoire virtuelle

Motivation

Que se passe-t-il si l'ensemble de l'espace d'adressage d'un processus ne rentre pas dans la mémoire physique ?

Peut-on éviter le gaspillage de la mémoire physique dû au fait qu'un processus n'utilise pas l'ensemble de son espace d'adressage à tout moment de sa durée de vie ?

Mémoire virtuelle

Motivation

Que se passe-t-il si l'ensemble de l'espace d'adressage d'un processus ne rentre pas dans la mémoire physique ?

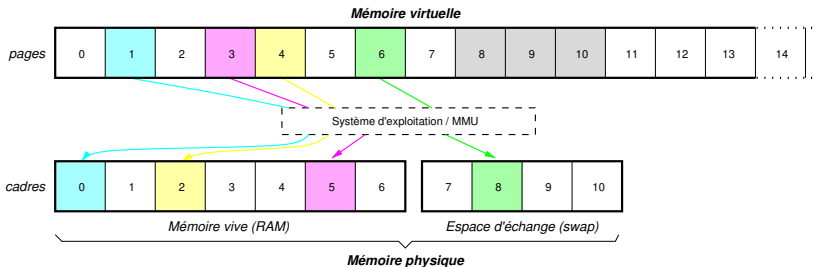
Peut-on éviter le gaspillage de la mémoire physique dû au fait qu'un processus n'utilise pas l'ensemble de son espace d'adressage à tout moment de sa durée de vie ?

- exposer au processus un espace mémoire virtuel considéré infini
- laisser le processus demander autant d'espace mémoire qu'il désire
- ne projeter une zone de la mémoire virtuelle en mémoire physique qu'à la première utilisation de celle-ci

Mémoire virtuelle

Pagination

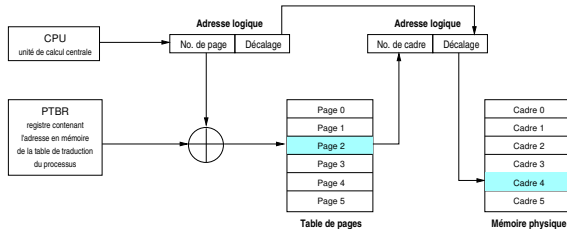
- découpage de l'espace mémoire en portions de taille 2^n (4 Kio le plus souvent)
 - mémoire physique découpée en **cadres**
 - mémoire virtuelle du processus découpée en **pages**
 - 1 cadre de la mémoire physique peut recevoir 1 page de la mémoire virtuelle
- projection de chaque page utilisée vers un cadre valide en mémoire physique implique l'unité de gestion de la mémoire (MMU) et le noyau du système d'exploitation



Mémoire virtuelle

Tables de traduction

- répertoires des correspondances *page-cadre* spécifiques à chaque processus
- conservées en mémoire physique
 - beaucoup de processus et de pages → consommation importante de la mémoire



Tables de traduction à niveaux multiples

- évincer des pages inutilisées sur le disque pour réduire la consommation de la mémoire

Mémoire virtuelle

Défaut de page

- référence vers une page qui ne se trouve pas en mémoire (premier accès à une page nouvellement allouée, ...)
- interruption du noyau
 - 1 charger la page manquante en mémoire
 - 2 refaire l'instruction concernée
- coût important pour le noyau du système d'exploitation et l'unité de gestion de la mémoire

Translation Lookaside Buffer (TLB)

- cache des correspondances *page-cadre* pour réduire le coût de la traduction des accès

Allocation

Types

- statique
- automatique (variables locales des fonctions)
- dynamique

Allocation dynamique

1 bas niveau

- allocation de pages
- projections entre la mémoire virtuelle et la mémoire physique
- appels système très coûteux → à minimiser

2 haut niveau

- répartir les allocations sur les pages déjà allouées
- différentes politiques d'allocation (*first fit*, *next fit*, *best fit*, ...)

Allocation

Copy-On-Write (COW)

- pas de duplication immédiate suite à un fork
- duplication à l'occasion de la première écriture (par le processus parent ou un des processus fils)

```
#include <unistd.h>
#include <stdio.h>
#include <sys/wait.h>
int main(void) {
    int val = 2;
    pid_t child = fork(); // pas de duplication à cet instant
    if(child == 0) {
        int local = 1;
        val += local; // duplication (première écriture)
    }
    else { wait(NULL); }
    printf("PID %d PPID %d VAL is %d\n", getpid(), getppid(), val);
    return 0;
}
```

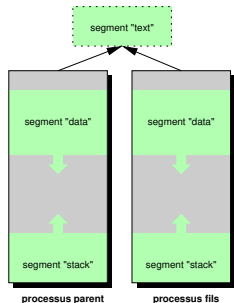
PID 11351 PPID 11345 VAL is 3

PID 11345 PPID 4190 VAL is 2

Partage de la mémoire

1 segment *text*

- code du programme
- en lecture seule
- lors d'un fork, ...



2 données entre processus ou fils d'exécution (lecture-écriture)

```
#define _GNU_SOURCE
#include <unistd.h>
#include <stdio.h>
#include <pthread.h>

void * work(void * arg) {
    int * val = (int *) arg;
    printf("TID %d VAL is %d (addr. %p)\n", getpid(), *val, val);
}

int main(int argc, char ** argv) {
    int val = 2; pthread_t tid;
    pthread_create(&tid, NULL, work, (void *) &val);
    printf("TID %d VAL is %d (addr. %p)\n", getpid(), val, &val);
    pthread_join(tid, NULL); return 0;
}
```

TID 18429 VAL is 2 (addr. 0x7ffe4e0bc8cc)

TID 18435 VAL is 2 (addr. 0x7ffe4e0bc8cc)